

## Data-Driven Resume Matching and Learning Recommendations for Top Companies

**Purpose:** To evaluate a candidate's resume in a structured and objective manner, identify how well it aligns with specific job profiles, and provide actionable insights to improve career readiness and employability.

### Objective:

1. Extract key information from a resume, including education, experience, skills, and certifications.
2. Compare the resume against job requirements for roles like Business Analyst, Data Analyst, and Data Scientist.
3. Quantify resume-job fit through compatibility scores and section-wise analysis.
4. Identify missing skills, qualifications, or experience and provide clear recommendations for improvement.
5. Enable candidates to tailor their resumes effectively to meet the expectations of top companies.

```
[ ]: !pip install python-docx PyPDF2
```

```
Collecting python-docx
```

```
  Downloading python_docx-1.2.0-py3-none-any.whl.metadata (2.0 kB)
```

```
Collecting PyPDF2
```

```
  Downloading pypdf2-3.0.1-py3-none-any.whl.metadata (6.8 kB)
```

```
Requirement already satisfied: lxml>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from python-docx) (6.0.2)
```

```
Requirement already satisfied: typing_extensions>=4.9.0 in /usr/local/lib/python3.12/dist-packages (from python-docx) (4.15.0)
```

```
Downloading python_docx-1.2.0-py3-none-any.whl (252 kB)
```

```
253.0/253.0 kB
```

```
14.7 MB/s eta 0:00:00
```

```
Downloading pypdf2-3.0.1-py3-none-any.whl (232 kB)
```

```
232.6/232.6 kB
```

```
25.5 MB/s eta 0:00:00
```

```
Installing collected packages: python-docx, PyPDF2
```

```
Successfully installed PyPDF2-3.0.1 python-docx-1.2.0
```

```

=====

[ ]: uploaded_jd = files.upload()
jd_file_name = list(uploaded_jd.keys())[0]

job_description_text = ""

if jd_file_name.endswith(".docx"):
    doc = Document(jd_file_name)
    for para in doc.paragraphs:
        job_description_text += para.text + "\n"

elif jd_file_name.endswith(".pdf"):
    with open(jd_file_name, "rb") as f:
        reader = PyPDF2.PdfReader(f)
        for page in reader.pages:
            job_description_text += page.extract_text() + "\n"

print(job_description_text[:500])

```

<IPython.core.display.HTML object>

Saving Shaileja Bsc data science 3rd year sem 6.docx to Shaileja Bsc data science 3rd year sem 6 (1).docx

Shaileja Karenolla

Shaileja Karenolla

Thane, Maharashtra.

+91-7738725514

shailejakarenolla123@gmail.com

LinkedIn: <http://www.linkedin.com/in/shaileja-karenolla-1ba523289>

GitHub: <https://github.com/shaileja-15>

Career Objective

Motivated and detail-oriented Data Analyst with hands-on experience in data visualization, SQL analytics, dashboard creation, and predictive modelling. Skilled in Excel, SQL, Python, Power BI, Tableau, and foundational Machine Learning. Capable of converting raw da

```

=====

[ ]: import re
import pandas as pd

resume_text = document_text.lower()
extracted_data = []

# Education
edu = re.findall(r"(b\?.?tech|b\?.?sc|b\?.?com|mba|m\?.?sc|bachelor|master|phd)",_
↪resume_text)

```

```

extracted_data.append(["Education", "", ".join(set(edu)) if edu else "Not_
↳Found"])

# Experience
exp = re.findall(r"\d+\+?\s*(years|yrs)", resume_text)
extracted_data.append(["Experience", "", ".join(set(exp)) if exp else "Not_
↳Found"])

# Skills
skills = re.findall(
    r"(python|sql|excel|power bi|tableau|machine learning|data_
↳analysis|statistics|finance|accounting)",
    resume_text
)
extracted_data.append(["Skillset", "", ".join(set(skills)) if skills else "Not_
↳Found"])

# Email & Phone
email = re.findall(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}",
↳resume_text)
phone = re.findall(r"\b\d{10}\b", resume_text)

extracted_data.append(["Email", email[0] if email else "Not Found"])
extracted_data.append(["Phone", phone[0] if phone else "Not Found"])

cell3_df = pd.DataFrame(extracted_data, columns=["Key", "Value"])
cell3_df

```

```

[ ]:      Key                                     Value
0  Education                                b.com, bachelor, mba
1  Experience                                Not Found
2  Skillset  excel, power bi, data analysis, tableau, machi...
3  Email                                     shailejakarenolla123@gmail.com
4  Phone                                     7738725514

```

```

[ ]: def extract_sections(text):
    return {
        "Education": re.findall(r"(b\.?tech|b\.?sc|b\.?com|mba|m\.?
↳sc|bachelor|master|phd)", text),
        "Experience": re.findall(r"\d+\+?\s*(years|yrs)", text),
        "Skillset": re.findall(
            r"(python|sql|excel|power bi|tableau|machine learning|data_
↳analysis|statistics|finance|accounting)",
            text
        ),
    }

```

```

        "Other Details": re.findall(
            r"(communication|analysis|problem|business|reporting|stakeholder)",
            text
        )
    }

resume_sections = extract_sections(resume_text)
jd_sections = extract_sections(job_description_text.lower())

def match_percentage(resume_list, jd_list):
    if not jd_list:
        return 0
    return round(len(set(resume_list) & set(jd_list)) / len(set(jd_list)) * 100, 2)

```

```

[ ]: from sentence_transformers import SentenceTransformer

bert_model = SentenceTransformer("bert-base-uncased")

bert_results = []
for key in resume_sections:
    bert_results.append([key, match_percentage(resume_sections[key],
jd_sections[key])])

bert_df = pd.DataFrame(bert_results, columns=["Parameter", "Matching %"])
bert_df

```

WARNING:torchao.kernel.intmm:Warning: Detected no triton, on systems without Triton certain kernels will not work

/usr/local/lib/python3.12/dist-packages/huggingface\_hub/utils/\_auth.py:94:

UserWarning:

The secret `HF\_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as secret in your Google Colab and restart your session.

You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn(

WARNING:sentence\_transformers.SentenceTransformer:No sentence-transformers model found with name bert-base-uncased. Creating a new one with mean pooling.

config.json: 0%| | 0.00/570 [00:00<?, ?B/s]

model.safetensors: 0%| | 0.00/440M [00:00<?, ?B/s]

tokenizer\_config.json: 0%| | 0.00/48.0 [00:00<?, ?B/s]

vocab.txt: 0%| | 0.00/232k [00:00<?, ?B/s]

tokenizer.json: 0%| | 0.00/466k [00:00<?, ?B/s]

```
[ ]:      Parameter  Matching %
0      Education      100.0
1      Experience       0.0
2      Skillset       100.0
3 Other Details      100.0
```

```
[ ]: roberta_model = SentenceTransformer("roberta-base")

roberta_results = []
for key in resume_sections:
    roberta_results.append([key, match_percentage(resume_sections[key],
jd_sections[key])])

roberta_df = pd.DataFrame(roberta_results, columns=["Parameter", "Matching %"])
roberta_df
```

WARNING:sentence\_transformers.SentenceTransformer:No sentence-transformers model found with name roberta-base. Creating a new one with mean pooling.

config.json: 0%| | 0.00/481 [00:00<?, ?B/s]

model.safetensors: 0%| | 0.00/499M [00:00<?, ?B/s]

Some weights of RobertaModel were not initialized from the model checkpoint at roberta-base and are newly initialized: ['pooler.dense.bias', 'pooler.dense.weight']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

tokenizer\_config.json: 0%| | 0.00/25.0 [00:00<?, ?B/s]

vocab.json: 0%| | 0.00/899k [00:00<?, ?B/s]

merges.txt: 0%| | 0.00/456k [00:00<?, ?B/s]

tokenizer.json: 0%| | 0.00/1.36M [00:00<?, ?B/s]

```
[ ]:      Parameter  Matching %
0      Education      100.0
1      Experience       0.0
2      Skillset       100.0
3 Other Details      100.0
```

```
[ ]: distilbert_model = SentenceTransformer("distilbert-base-uncased")

distilbert_results = []
for key in resume_sections:
    distilbert_results.append([key, match_percentage(resume_sections[key],
    ↪jd_sections[key])])

distilbert_df = pd.DataFrame(distilbert_results, columns=["Parameter",
    ↪"Matching %"])
distilbert_df
```

WARNING:sentence\_transformers.SentenceTransformer:No sentence-transformers model found with name distilbert-base-uncased. Creating a new one with mean pooling.

```
config.json: 0%|          | 0.00/483 [00:00<?, ?B/s]
model.safetensors: 0%|          | 0.00/268M [00:00<?, ?B/s]
tokenizer_config.json: 0%|          | 0.00/48.0 [00:00<?, ?B/s]
vocab.txt: 0%|          | 0.00/232k [00:00<?, ?B/s]
tokenizer.json: 0%|          | 0.00/466k [00:00<?, ?B/s]
```

```
[ ]:      Parameter  Matching %
0      Education      100.0
1      Experience       0.0
2      Skillset      100.0
3  Other Details      100.0
```

=====

```
[ ]: # Top 15 companies JD-based Job Profiles
business_analyst_jd = """
Business Analyst role (Top Companies): SQL, Excel, Power BI, stakeholder
    ↪communication,
requirement gathering, business analysis, documentation, reporting. Bachelor
    ↪degree required.
"""

data_analyst_jd = """
Data Analyst role (Top Companies): SQL, Python, Excel, Power BI, Tableau,
data cleaning, data analysis, statistics, dashboard creation, reporting,
    ↪business insights.
"""

data_scientist_jd = """
Data Scientist role (Top Companies): Python, Machine Learning, Statistics,
Data Modeling, Problem Solving, Communication, Analytics, Project Experience.
Master's preferred.
```

```
"""
```

```
[ ]: profiles = {
    "Business Analyst": business_analyst_jd,
    "Data Analyst": data_analyst_jd,
    "Data Scientist": data_scientist_jd
}

final_results = []

for role, jd in profiles.items():
    jd_sec = extract_sections(jd.lower())
    scores = [
        match_percentage(resume_sections[k], jd_sec[k])
        for k in resume_sections
    ]
    final_results.append([role, round(sum(scores)/len(scores), 2)])

final_df = pd.DataFrame(final_results, columns=["Job Role", "Overall Match %"])
final_df
```

```
[ ]:
      Job Role  Overall Match %
0  Business Analyst          70.0
1    Data Analyst          50.0
2  Data Scientist          50.0
```

```
[ ]: # Improved missing skills + resume improvement

def extract_sections(text):
    return {
        "Education": re.findall(r"(b\.?tech|b\.?sc|b\.?com|mba|m\.?
↪sc|bachelor|master|phd)", text.lower()),
        "Experience": re.findall(r"\d+\+?\s*(years|yrs)", text.lower()),
        "Skillset": re.findall(
            r"(python|sql|excel|power bi|tableau|machine learning|data_
↪analysis|statistics|finance|accounting)",
            text.lower()
        ),
        "Other Details": re.findall(
            r"(communication|analysis|problem|business|reporting|stakeholder)",
            text.lower()
        )
    }
```

```

resume_sections = extract_sections(document_text)

profiles = {
    "Business Analyst": business_analyst_jd,
    "Data Analyst": data_analyst_jd,
    "Data Scientist": data_scientist_jd
}

improvement_results = []

for role, jd in profiles.items():
    jd_sec = extract_sections(jd)
    missing_skills = set(jd_sec["Skillset"]) - set(resume_sections["Skillset"])
    missing_edu = set(jd_sec["Education"]) - set(resume_sections["Education"])
    missing_exp = set(jd_sec["Experience"]) - set(resume_sections["Experience"])
    missing_other = set(jd_sec["Other Details"]) - set(resume_sections["Other_
↵Details"])

    # Combine suggestions
    suggestions = []
    if missing_skills: suggestions.append("Skills: " + ", ".
↵join(missing_skills))
    if missing_edu: suggestions.append("Education: " + ", ".join(missing_edu))
    if missing_exp: suggestions.append("Experience: " + ", ".join(missing_exp))
    if missing_other: suggestions.append("Other Details: " + ", ".
↵join(missing_other))
    if not suggestions: suggestions.append("None - Resume fully matches this JD!
↵")

    improvement_results.append([role, "; ".join(suggestions)])

improvement_df = pd.DataFrame(improvement_results, columns=["Job Role", "Resume_
↵Improvement Suggestions"])
improvement_df

```

```

[ ]:
      Job Role      Resume Improvement Suggestions
0  Business Analyst      Other Details: stakeholder
1    Data Analyst  None - Resume fully matches this JD!
2    Data Scientist      Education: master

```

```

[ ]: import re
      import pandas as pd

      def extract_sections(text):
          return {

```



```

        "Education": re.findall(r"(b\.?tech|b\.?sc|b\.?com|mba|m\.?
↪sc|bachelor|master|phd)", text.lower()),
        "Experience": re.findall(r"\d+\+?\s*(years|yrs)", text.lower()),
        "Skillset": re.findall(
            r"(python|sql|excel|power bi|tableau|machine learning|data_
↪analysis|statistics|finance|accounting)",
            text.lower()
        ),
        "Other Details": re.findall(
            r"(communication|analysis|problem|business|reporting|stakeholder)",
            text.lower()
        )
    }

def calculate_percentage(resume_list, jd_list):
    if not jd_list:
        return 100 # if JD doesn't require anything, consider full match
    matched = set(resume_list) & set(jd_list)
    return round(len(matched) / len(set(jd_list)) * 100, 2)

def calculate_missing(resume_list, jd_list):
    return set(jd_list) - set(resume_list)

resume_sections = extract_sections(document_text)

# Job profiles
profiles = {
    "Business Analyst": business_analyst_jd,
    "Data Analyst": data_analyst_jd,
    "Data Scientist": data_scientist_jd
}

results = []

for role, jd in profiles.items():
    jd_sec = extract_sections(jd)

    # Calculate percentages
    perc_edu = calculate_percentage(resume_sections["Education"],
↪jd_sec["Education"])
    perc_exp = calculate_percentage(resume_sections["Experience"],
↪jd_sec["Experience"])
    perc_skill = calculate_percentage(resume_sections["Skillset"],
↪jd_sec["Skillset"])
    perc_other = calculate_percentage(resume_sections["Other Details"],
↪jd_sec["Other Details"])

```

```

overall_match = round((perc_edu + perc_exp + perc_skill + perc_other)/4, 2)

# Missing items
missing_skills = calculate_missing(resume_sections["Skillset"],
↪jd_sec["Skillset"])
missing_edu = calculate_missing(resume_sections["Education"],
↪jd_sec["Education"])
missing_exp = calculate_missing(resume_sections["Experience"],
↪jd_sec["Experience"])
missing_other = calculate_missing(resume_sections["Other Details"],
↪jd_sec["Other Details"])

suggestions = []
if missing_skills: suggestions.append("Skills: " + ", ".
↪join(missing_skills))
if missing_edu: suggestions.append("Education: " + ", ".join(missing_edu))
if missing_exp: suggestions.append("Experience: " + ", ".join(missing_exp))
if missing_other: suggestions.append("Other Details: " + ", ".
↪join(missing_other))
if not suggestions: suggestions.append("None - Resume fully matches this JD!
↪")

results.append([
    role, perc_edu, perc_exp, perc_skill, perc_other, overall_match, "; ".
↪join(suggestions)
])

final_summary_df = pd.DataFrame(results, columns=[
    "Job Role", "Education %", "Experience %", "Skillset %", "Other %",
↪"Overall Match %", "Resume Improvement Suggestions"
])

final_summary_df

```

```

[ ]:
      Job Role  Education %  Experience %  Skillset %  Other % \
0  Business Analyst      100.0         100       100.0      80.0
1    Data Analyst      100.0         100       100.0     100.0
2  Data Scientist       0.0         100       100.0     100.0

      Overall Match %      Resume Improvement Suggestions
0              95.0      Other Details: stakeholder
1             100.0  None - Resume fully matches this JD!
2             75.0      Education: master

```

**Conclusion:**

This project offers a data-driven approach to assess how well a resume aligns with various job roles required by top companies by systematically comparing resume content with role-specific job descriptions. It provides clear insights into a candidate's strengths, identifies gaps in skills, education, and experience, and calculates role-wise matching percentages to determine the best career fit. In addition to evaluation, the project delivers actionable learning recommendations that guide candidates on what to improve or acquire to enhance their employability. Overall, it serves as an effective resume assessment and career planning tool, helping candidates align their profiles with industry expectations and prepare more confidently for competitive recruitment processes.

=====